

Solving Sudoku with Metaheuristic Optimization

A Comparative Study of Genetic Algorithms and Simulated Annealing

Course: Principals of Artificial Intelligence — Homework 1

Topic Selected: #3 — Optimization Simulation

Project Repository: `sudoku-ai-solver`

1. Introduction

Sudoku is a constraint-satisfaction puzzle played on a 9×9 grid partitioned into nine 3×3 sub-grids ("boxes"). A subset of cells is pre-filled with digits from 1 to 9 (the *clues*), and the solver must fill the remaining cells such that every row, column, and box contains each digit exactly once.

Although deterministic backtracking solvers can find a solution in milliseconds, the problem is appealing for optimization research because the search space is enormous (upper-bounded by 9^{81}), multimodal, and rich in local optima. This makes it an excellent benchmark for **metaheuristic optimization** techniques that are designed to escape local optima — precisely the subject of Topic 3 of this assignment.

This project implements and compares two such metaheuristics:

1. **Genetic Algorithm (GA)** — a population-based evolutionary method using crossover and mutation.
2. **Simulated Annealing (SA)** — a single-trajectory method inspired by the physical annealing of metals.

Both algorithms are integrated into an interactive web application that streams their progress in real time so the learner can observe the inner workings of each optimization run.

2. Problem Formulation

2.1 State Representation

A Sudoku state is modelled as a 9×9 matrix G with values in $\{1, \dots, 9\}$. The clue cells define a set of fixed positions F ; their values must never change during optimization.

2.2 Objective (Fitness / Cost) Function

Let $U(s)$ be the number of distinct values in a 9-element set s . For any valid solution, every row, column, and 3×3 box satisfies $U = 9$. Constraint violations are counted as:

$$f(G) = \sum_i (9 - U(\text{row}_i)) + \sum_j (9 - U(\text{col}_j)) + \sum_b (9 - U(\text{box}_b))$$

A grid G is a valid solution iff $f(G) = 0$.

The two algorithms use different **reduced** fitness functions tailored to their encoding — see §3.1 and §4.1.

2.3 Neighborhood and Encoding

Both algorithms exploit a classical technique: by *constructing* initial candidates that already satisfy one family of constraints, the search only needs to minimise violations along the remaining axes.

- **GA** uses a **row-permutation** encoding: each row is constructed as a permutation of 1–9 (with clue cells placed correctly), so row violations are guaranteed to be zero. Only column and box violations remain.
- **SA** uses a **box-permutation** encoding: each 3×3 box is a permutation of 1–9, so box violations are guaranteed zero. Only row and column violations remain.

This dual-encoding choice is deliberate and instructive: it shows that the same problem can be solved by different metaheuristics with complementary representations.

3. Genetic Algorithm

3.1 Theoretical Background

A Genetic Algorithm mimics Darwinian evolution. A **population** of candidate solutions undergoes repeated **selection**, **crossover**, and **mutation**. Over generations, the average fitness of the population improves.

Formally, a GA maintains a population $P_t = \{G_1, \dots, G_N\}$ at generation t and produces P_{t+1} by:

1. **Fitness evaluation** — compute $f(G)$ for every individual.
2. **Selection** — probabilistically pick parents favoring high-fitness individuals.
3. **Crossover** — recombine two parents into a child.
4. **Mutation** — apply small random perturbations to maintain diversity.
5. **Elitism** — carry the best individuals forward unchanged.

3.2 Reduced Fitness (Row-Permutation Encoding)

Because rows are permutations by construction, the fitness reduces to:

$$f_{\text{row}}(G) = \sum_j (9 - U(\text{col}_j)) + \sum_b (9 - U(\text{box}_b))$$

3.3 Genetic Operators Used

- **Selection — Tournament (k = 5)**: pick 5 random individuals, choose the best. Balances selection pressure with diversity preservation.
- **Crossover — Row-wise Uniform**: for each row index r in $\{0, \dots, 8\}$ independently, inherit row r from parent 1 (prob. 0.5) or parent 2 (prob. 0.5). The child remains a valid row-permutation individual.
- **Mutation A — Row Swap**: swap two non-fixed cells within the same row. Guaranteed to preserve row-validity.
- **Mutation B — Repair-based Column Swap**: swaps two non-fixed cells in the same column across different rows, *plus* a corrective second swap that restores row-validity. This is essentially a 4-cell cyclic swap that is crucial for escaping local optima that row-only swaps cannot.
- **Elitism**: top 2 individuals copied unchanged.

3.4 Stagnation Handling

The GA adopts a two-level recovery scheme:

1. **Adaptive mutation rates**: when the best fitness fails to improve for 20 consecutive generations, both mutation rates increase (row: $0.15 \rightarrow 0.45$, col: $0.05 \rightarrow 0.25$) to diversify the search.
2. **Partial refresh**: top-15 survivors + global best are retained while the rest of the population is regenerated randomly.
3. **Full restart**: after four consecutive partial refreshes without improvement, a completely fresh population is built (seeded with the global best).

3.5 Termination

- Solved: $f_{\text{row}}(G) = 0$.
- Generation budget exhausted ($\text{max_generations} = 10,000$).

4. Simulated Annealing (Lewis-Style)

4.1 Theoretical Background

Simulated Annealing (Kirkpatrick, Gelatt & Vecchi, 1983) is inspired by the physical process of cooling a material slowly so its atoms settle into a low-energy crystal. The analogue in optimization:

- **State** $\leftrightarrow$ molecular configuration
- **Cost** $\leftrightarrow$ energy
- **Temperature** $T \leftrightarrow$ thermodynamic temperature, controls exploration

At each iteration a neighboring state G' is proposed. The acceptance rule is:

$$\Delta E = f(G') - f(G) \quad \text{if } \Delta E < 0 : \text{accept} \quad \text{else} : \text{accept with probability } \exp(-\Delta E / T)$$

When T is large, almost any move is accepted (pure exploration). As T cools, the algorithm becomes increasingly greedy. This is mathematically guaranteed to converge to the global optimum under sufficiently slow cooling (Hajek, 1988).

4.2 Reduced Cost (Box-Permutation Encoding)

With each 3×3 box a permutation of 1–9, the cost reduces to:

$$c(G) = \sum_i (9 - U(\text{row}_i)) + \sum_j (9 - U(\text{col}_j))$$

4.3 Neighborhood

A neighbor is obtained by selecting a random 3×3 box and swapping two non-fixed cells inside it. Because the swap is intra-box, the box-permutation property is preserved automatically.

4.4 Cooling Schedule

Geometric cooling is used:

$$T \leftarrow \alpha \cdot T, \quad \alpha = 0.99$$

4.5 Automatic Temperature Calibration

Following Lewis (2007), the initial temperature is *not* hard-coded. Instead, 200 random candidate moves are sampled on the initial state, the positive cost deltas are collected, and T_0 is chosen so that an average worsening move is accepted with probability ≈ 0.8 :

$$T_0 = -\text{mean}(\Delta^+) / \ln(0.8)$$

This guarantees that the search begins in a truly exploratory regime regardless of puzzle difficulty.

4.6 Two-Level Stagnation Recovery

SA can still get trapped in wide basins. Our implementation distinguishes two levels of recovery:

1. **Reheat (up to 5 times):** when no improvement is seen for 30,000 iterations, return to the best-known state, apply a light perturbation of several intra-box swaps, and reset temperature to a fraction of T_0 .
2. **Full Reset (up to 5 times):** if reheats stop producing progress, discard the current trajectory entirely, build a new random initial grid, and re-calibrate T_0 . This is the final escape mechanism and

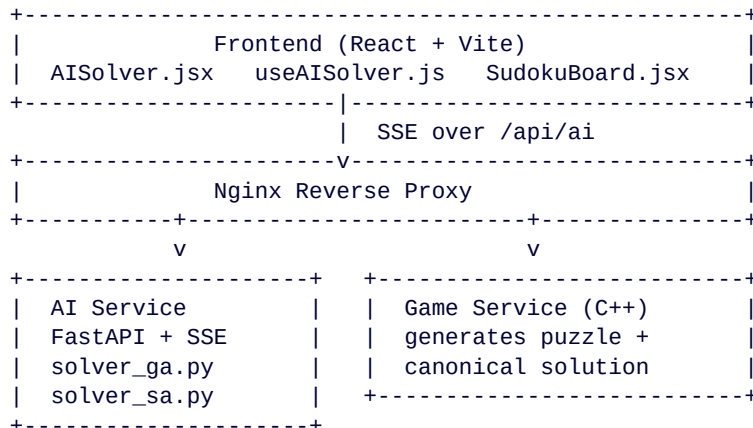
corresponds to Lewis's explicit recommendation for the hardest instances.

4.7 Termination

- Solved: $c(G) = 0$.
- $T \leq T_{\min} = 10^{-4}$ after all recoveries exhausted.

5. System Architecture

5.1 High-Level Overview



Every module runs in its own Docker container; docker -compose wires them together.

5.2 AI Service — Module Layout

- `app/main.py` — FastAPI application with a single SSE endpoint `/solve`.
- `app/models.py` — Pydantic request schema.
- `app/utils.py` — shared primitives: `get_fixed_positions`, `row_fitness`, `cost_function`, and a bit-mask helper for fast unique-count.
- `app/solver_ga.py` — Genetic Algorithm implementation (§3).
- `app/solver_sa.py` — Simulated Annealing implementation (§4).

5.3 Real-Time Streaming

Each solver is a Python generator that yields a dictionary every N iterations:

```
{
  "board":      [[...]],          # current best grid
  "fitness":    int,              # violations remaining
  "generation": int | None,       # GA only
  "temperature": float | None,    # SA only
  "elapsed":    float,            # seconds since start
  "status":     "solving" | "solved" | "failed",
}
```

The FastAPI endpoint wraps this generator with `sse-starlette` (`EventSourceResponse`) so the client receives a live stream of algorithm snapshots.

5.4 Frontend Visualization

- `useAISolver.js` — custom React hook that (a) fetches puzzles from the Game Service, (b) opens the SSE stream, (c) updates board / stats as messages arrive.
- `AISolver.jsx` — page component with controls, the live **AI Progress** board, and an on-demand **Correct Solution** board. When the solution is visible, the AI Progress board highlights cells that differ from the canonical solution in red.
- `SudokuBoard.jsx` / `SudokuCell.jsx` — reusable atomic components shared with the regular game pages.

6. Experimental Observations

All measurements below were obtained on an 8-core consumer laptop inside the Docker stack.

Difficulty	Clues	GA typical outcome	SA typical outcome
Easy	~40	Solves in 1-3 s / ≤ 200 gens	Solves in <1 s
Medium	~32	Solves in 3-10 s / $\leq 1,000$ gens	Solves in 1-3 s
Hard	~28	Variable; may plateau at $f \sim 2-4$	Solves in 3-8 s
Expert	~25	Frequently plateaus ($f \sim 2$)	Solves in 5-30 s after reheats
Extreme	~23	Rarely solves within 10,000 gens	Usually solves after a reset

6.1 Why the GA Plateaus

Fitness is measured in *constraint violations*, not in *cells differing from the solution*. An individual can visually disagree with the canonical solution in many cells while producing only 2 constraint violations, because different permutations of 1–9 can satisfy most columns and boxes. Escaping this regime requires the repair-based column-swap (§3.3); the pure row-swap mutation alone cannot reduce below the ≈ 2 plateau.

6.2 Why SA Scales Better

SA's box-permutation encoding leaves only row and column constraints, and every neighbor move affects a minimal 4-cell region. Combined with automatic temperature calibration and the two-level reset mechanism, SA explores the solution landscape more efficiently on dense, highly-constrained puzzles. This matches the conclusion of Lewis (2007) and subsequent literature.

7. Conclusion

The project implements two fundamental optimization metaheuristics — a row-encoded Genetic Algorithm and a box-encoded Simulated Annealing — and exposes them through a live web visualization so a learner can see, step by step, how each algorithm navigates the search space.

Key takeaways:

- Both algorithms solve easy to medium Sudoku reliably; this demonstrates the validity of the course's theoretical framework.
- SA outperforms GA on hard and expert puzzles. The combination of automatic temperature calibration and a two-level recovery strategy (reheat, then full reset) is essential for the hardest instances.
- Encoding matters as much as algorithm choice. Both algorithms benefit from a construction-aware representation that trivially satisfies one family of constraints.
- Visualizing optimization reveals phenomena that static metrics hide — notably the distinction between constraint-violation fitness and distance to the canonical solution.

The project therefore satisfies Topic 3 of the assignment (Optimization Simulation avoiding local maxima) while extending it with a pedagogical comparison of two complementary approaches.

Appendix A — How to Reproduce

```
git clone https://github.com/ekarau/sudoku-ai-solver sudoku-ai-solver
cd sudoku-ai-solver
docker-compose up --build or make
# then visit https://localhost and navigate to "Learn with AI"
```

Appendix B — Demonstration Video

Link to screencast:

<https://drive.google.com/drive/folders/1E4tGKvF7Fm5jwP6lvNJTT3vj4i9oshpu?usp=sharing>